

NAME : PRERANA MUDIAR
ROLL NO : CSB24021

```
import sys
import gc

# Node class
class Node:
    def __init__(self, name):
        self.name = name
        self.link = None

    def __del__(self):
        print(f"{self.name} is being destroyed")

# -----
# Step 1: Create two objects
# -----

A = Node("A")
B = Node("B")

# -----
# Step 2: Create circular reference
# -----

A.link = B
B.link = A

print("Cycle created:")
print("A ->", A.link.name)
print("B ->", B.link.name)

# -----
# Step 3: Check reference counts
# -----

print("\nReference Counts:")
print("A:", sys.getrefcount(A))
print("B:", sys.getrefcount(B))
```

```
# Save object IDs for investigation later
a_id = id(A)
b_id = id(B)

# -----
# Step 4: Delete original references
# -----

del A
del B

print("\nA and B variables deleted.")

# -----
# Step 5: Investigate memory
# -----

print("\nObjects still exist because of circular reference.")

found = False

for obj in gc.get_objects():
    if id(obj) == a_id or id(obj) == b_id:
        print("Found object still in memory:", obj.name)
        found = True

if not found:
    print("Objects not found.")

# -----
# Step 6: Force Garbage Collection
# -----

print("\nRunning Garbage Collector...")

unreachable = gc.collect()

print("Unreachable objects collected:", unreachable)

# -----
# Step 7: Verify cleanup
# -----
```

```
found_after = False

for obj in gc.get_objects():
    if id(obj) == a_id or id(obj) == b_id:
        found_after = True

if not found_after:
    print("Cycle successfully removed from memory.")
else:
    print("Objects still exist.")
```

OUTPUT:

```
Cycle created:
A -> B
B -> A

Reference Counts:
A: 3
B: 3

A and B variables deleted.

Objects still exist because of circular reference.
Found object still in memory: A
Found object still in memory: B

Running Garbage Collector...
A is being destroyed
B is being destroyed
Unreachable objects collected: 24
Cycle successfully removed from memory.

[Done] exited with code=0 in 0.084 seconds
```